Dynamic Programming

Sanghyeon Lee

2026/4/5

RL – Recap.

Reward Hypothesis

모든 목표는 누적 보상 의 최대화로 표현될 수 있다.

RL

어떤 행동을 해야 누적 보상을 최대화 할 수 있는가?

- 모델을 아는 경우 → DP
- 모르는 경우 → MC, TD, etc..

DP - 전제조건

Finite MDP

- State, Action, Reward set, and **Dynamics**
- $S, A, R, \quad \boldsymbol{p}(\boldsymbol{s}', \boldsymbol{r} \mid \boldsymbol{s}, \boldsymbol{a})$

대부분의 경우 DP 는 Finite MDP의 경우에 적용한다.

* 연속 공간의 경우 Quantization, Function Approximation (Cp 9) 를 사용하여 DP 적용 가능하지만, 정확한 해는 특수한 경우에만 얻을 수 있음

DP - 한계

대부분의 현실 RL 문제는 Dynamics를 정확히 알 수 없음
- 안다고 하더라도 모든 공간을 계산하는 것은 너무 비효율적

Q. 그럼 DP를 굳이 왜배우나요?

A. **DP는 근본**
- DP는 이후 모든 RL 기법들의 방법론적 기반이 됨
- DP에서 처음 등장한 개념들이 이후에도 계속 사용됨

DP – Value Function

Bellman Expectation Equation

$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_\pi(s'))$ - 정책 π를 따를 때 상태 s의 기대 보상

Bellman Optimality Equation

$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_*(s'))$ - 최적 정책을 따를 때 상태 s의 기대 보상

Key Idea - **Equation → Assignment**

DP – Value Function

Equation

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_*(s'))$$

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_\pi(s'))$$

Bellman Optimality Eq

$v_*(s)$라면 이 식을 만족해야 함

Assignments

$$v_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_k(s'))$$

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a)(r + \gamma v_k(s'))$$

Value Function을 업데이트하는 규칙

반복하면 $v_*(s), v_\pi(s)$에 수렴함

Index

1. Iterative Policy Evaluation
2. Policy Improvement
3. Policy Iteration
4. Value Iteration
5. Generalized Policy Iteration

Policy Evaluation

Policy Evaluation

- 임의의 정책 π가 주어졌을 때 v_π를 어떻게 계산하는가?

$v_\pi(s) = E[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$: |S|개 미지수에 대한 연립방정식

- 킹론상 Dynamics를 전부 알고 있다면 풀 수는 있음
- |S|가 커질수록 현실적으로 어려움

Iterative Policy Evaluation

$$v_\pi(s) = E[R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s]$$

$$\rightarrow v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r\,|s, a)\,(r + \gamma\, v_\pi(s'))$$

$$\rightarrow v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r\,|s, a)\,(r + \gamma\, v_k(s'))$$ - 1 Sweep

- $v_k = v_\pi$ 에서 업데이트 종료 (Fixed point)
- v_π의 존재가 보장되는 경우 $(\gamma < 1\ or$ 에피소드 종료 보장$)$, $k \rightarrow \infty$ 일 때 v_π 수렴성 보장

*Full backup

- 한 상태 s의 가치을 도달 가능한 모든 상태 s'의 현재 추정 가치를 기반으로 계산

Iterative Policy Evaluation

Termination Condition

$$\max_{s \in S} |v_{k+1}(s) - v_k(s)| < \theta$$

이론상 $k \to \infty$ 에서 수렴하기 때문에
θ에서 early stopping

Input π, the policy to be evaluated
Initialize an array $V(s) = 0$, for all $s \in \mathcal{S}^+$
Repeat
  $\Delta \leftarrow 0$
  For each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\left[r + \gamma V(s')\right]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)
Output $V \approx v_\pi$

Iterative Policy Evaluation - Implementation

Two-Array

- 배열이 두 개지요 (v_k, v_{k+1})
- 업데이트 순서가 $s_1, s_2, \ldots$ 라고 할 때 s_1이 업데이트 된 이후에도 $v_k(s_1)$를 사용
- 더 안전하고 병렬 연산 가능

In-Place

- 배열이 하나지요
- 업데이트 순서가 $s_1, s_2, \ldots$ 라고 할 때 s_1 업데이트 후, 바로 $v_{k+1}(s_1)$를 활용
- 일반적으로 더 빨리 수렴, 메모리 절반

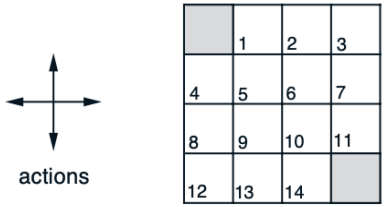
Iterative Policy Evaluation - Implementation

actions

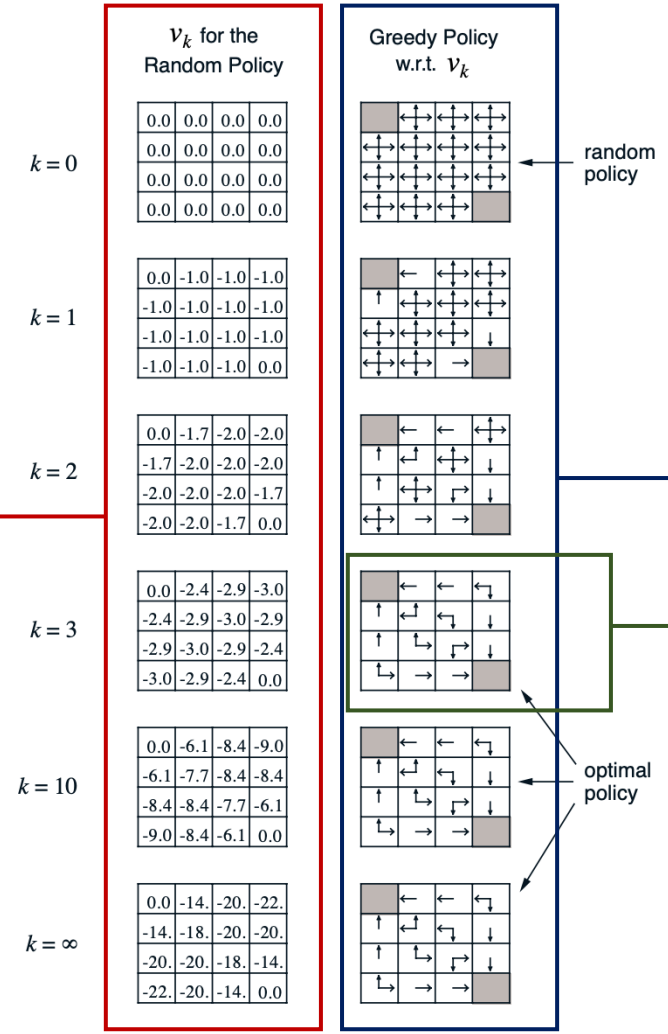
	1	2	3
4	5	6	7
8	9	10	11
12	13	14	

$R = -1$
on all transitions

Iterative Policy Evaluation - Implementation



$R = -1$
on all transitions



이게 Iterative Policy Evaluation

이건 아님

K = 3에서 이미
Greedy = Optimal
→ Value Iteration의 동기

Policy Improvement

Policy Evaluation 완료 → v_π 를 앎 → 현재 정책 π 를 개선할 수 있음

$v_\pi(s)$를 알 때, 상태 s에서 $\pi(s)$가 아닌 어떤 행동을 해야 하는가?

→ $q_\pi(s,a)$ 도입

$v_\pi(s)$	$q_\pi(s,a)$
- s 에서 π를 따를 때의 가치	- s 에서 행동 a를 한 번 하고 이후 π를 따를 때의 가치

$$q_\pi(s,a) = \sum_{s',r} p(s',r \mid s,a)\,[r + \gamma v_\pi(s')]$$

$q_\pi(s,a) > v_\pi(s) \rightarrow a$ 를 따르는 게 낫다

$q_\pi(s,a) \leq v_\pi(s) \rightarrow \pi(s)$ 를 그냥 따르는 게 낫다

Policy Improvement

Policy Improvement Theorem

- 모든 s에 대해 $q_\pi\big(s, \pi'(s)\big) \geq v_\pi(s)$ → 모든 s에 대해 $v_{\pi'}(s) \geq v_\pi(s)$

Proof

$$
\begin{aligned}
&v_\pi(s) \leq q_\pi\big(s, \pi'(s)\big) = E_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\leq E_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\
&= E_{\pi'}[R_{t+1} + \gamma E_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2})] \mid \mid S_t = s] \\
&= E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s] \\
&\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s] \leq \ \dots \\
&\leq E_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \ \dots \mid S_t = s] = v_{\pi'}(s)
\end{aligned}
$$

Policy Improvement

Greedy Policy

- $\pi'(s) = \underset{a}{\operatorname{argmax}}\, q_\pi(s,a) = \underset{a}{\operatorname{argmax}} \sum_{s',r} p(s',r \mid s,a)\,(r + \gamma v_\pi(s'))$

모든 행동 중 v_π 기준으로 한 스텝 앞만 보고 최선 선택

- $q_\pi\big(s,\pi'(s)\big) = \max_a q_\pi(s,a) \geq q_\pi\big(s,\pi(s)\big) = v_\pi(s)$

→ 항상 Theorem을 만족

Policy Improvement

$\pi' = \pi$인 경우

$v_\pi(s) = \max_a \sum_{s',r} p(s', r \mid s, a)\,(r + \gamma v_\pi(s))$ ← Bellman Optimality Eq

$v_\pi = v_*$

Policy Improvement 는 항상 strictly better한 π'를 만들거나 이미 최적이거나 둘 중 하나

Policy Iteration

Policy Evaluation → v_π 계산

Policy Improvement → v_π로 더 나은 정책 π' 만들기

→번갈아 반복한다면?

$\pi_0 \rightarrow E \rightarrow v_{\pi_0} \rightarrow I \rightarrow \pi_1 \rightarrow E \rightarrow v_{\pi_1} \rightarrow I \ \ldots$

→ 매 사이클 정책이 단조 개선 (monotonically improving)

Policy Iteration

항상 수렴하는가? → Yes

DP는 Finite MDP를 가정

→가능한 policy 의 수는 유한 (최대 $|A|$^$|S|$개)

→매 iteration: strictly better한 정책 or 최적 달성

→같은 정책이 두 번 등장 불가능

→유한 iteration 안에 수렴 → 수렴 == 최적

Policy Iteration

1. V(s) 초기화

2. Policy Evaluation

현재 정책에 대해 v 수렴할 때 까지 반복

3. Policy Improvement

Greedy 업데이트
Policy-stable → 종료
Else → 반복

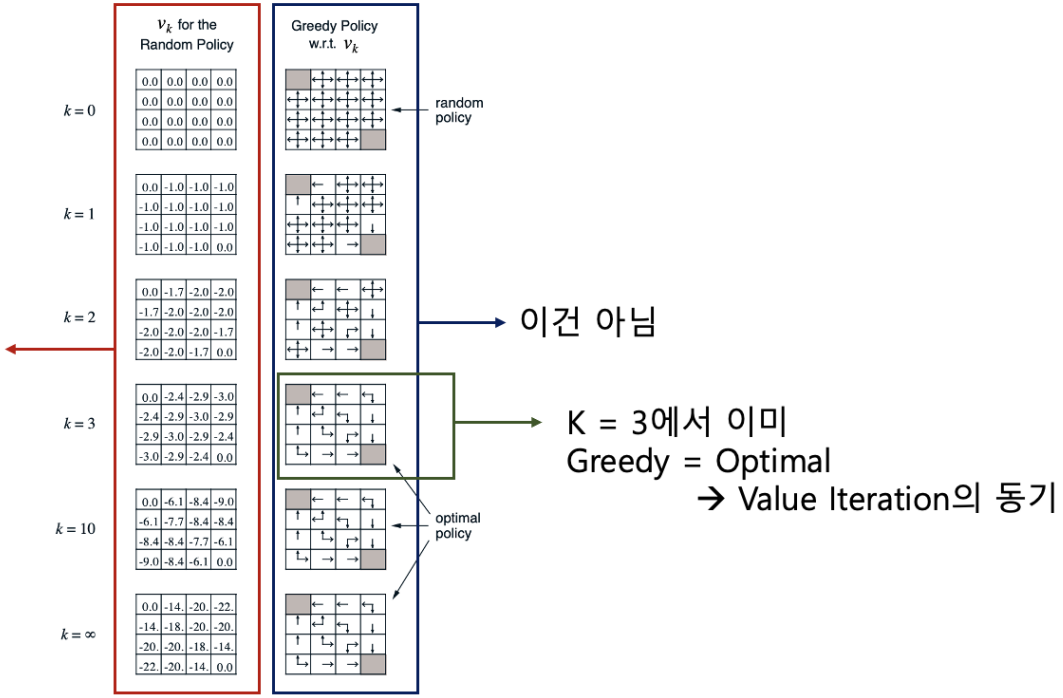
* 이론상 무한반복 ㄱㄴ인데 귀찮아서 이대로 썼음

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Repeat
   $\quad \Delta \leftarrow 0$
   $\quad$ For each $s \in \mathcal{S}$:
   $\quad\quad v \leftarrow V(s)$
   $\quad\quad V(s) \leftarrow \sum_{s',r} p(s', r|s, \pi(s))\big[r + \gamma V(s')\big]$
   $\quad\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $\quad a \leftarrow \pi(s)$
   $\quad \pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
   $\quad$ If $a \neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return V and π; else go to 2

Value Iteration



$$\pi_0 \rightarrow \boxed{E} \rightarrow v_{\pi_0} \rightarrow I \rightarrow \pi_1 \rightarrow E \rightarrow v_{\pi_1} \rightarrow I \ \ldots$$

Policy Evaluation → 수렴 시 까지 반복 but

이걸 꼭 수렴할 때 까지 해야 하나?

완전한 Policy Evaluation은 계산 자원의 낭비
입맛대로 Truncate해도 Policy iteration의 수렴성은 보장된다

Value Iteration → 1 sweep씩만 Evaluation하면서 반복하는 방법

Value Iteration

Policy Evaluation 1 sweep

$$v_{k+1}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r \mid s, a)\,(r + \gamma\, v_k(s'))$$

Policy Improvement

$$\pi'(s) = \operatorname*{argmax}_{a} \sum_{s',r} p(s', r \mid s, a)\,(r + \gamma v_\pi(s'))$$

→ $v_{k+1}(s) = \max_{a} \sum_{s',r} p(s', r \mid s, a)\,(r + \gamma\, v_k(s'))$

Bellman Optimality Eq를 Update rule로 바꾼것과 동치

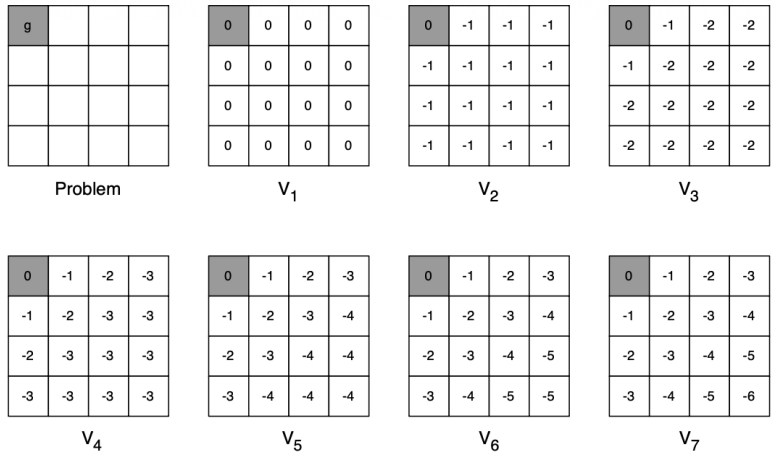
- Policy Evaluation 은 v_π를 향해, Value Iteration은 v_*을 향해 수렴한다.

Value Iteration

	Policy Iteration	**Value Iteration**
Iteration 수	적음 (보다 확실한 정책 개선)	많을 수 있음
Iteration 비용	무거움 (Evaluation 완전수렴)	가벼움 (1 sweep)
Policy 추출	매 Iteration	수렴 후 1번 (그 전까지는 V만 업데이트)

우열 X, 문제에 따라 두 방법 모두 널리 사용됨

Value Iteration



*이 방식은 Two-Array임

종료 조건은 Policy Evaluation 과 동일

Initialize array V arbitrarily (e.g., $V(s) = 0$ for all $s \in \mathcal{S}^+$)

Repeat
$\quad \Delta \leftarrow 0$
$\quad$ For each $s \in \mathcal{S}$:
$\qquad v \leftarrow V(s)$
$\qquad V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
$\qquad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$ (a small positive number)

Output a deterministic policy, π, such that
$\quad \pi(s) = \operatorname{argmax}_a \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$

Figure 4.5: Value iteration.

Asynchronous DP

백개먼: 서양 전통놀이, 10^20개의 State 존재

- 초당 100만번 업뎃해도 1sweep = 3000만 년

→ 모든 상태를 매번 업데이트하지 않아도 괜찮다.

Asynchronous DP

- 어떤 순서로든 업데이트해도 된다.
    - Sync : s1 → s2 → s3 → …
    - Async: s2 → s4 → s1 → …

- 어떤 순서를 더 자주 업데이트해도 된다. (빈도 자우, **but 빈도 0은 안됨**)

수렴 조건

모든 상태가 무한히 자주 업데이트되어야 함 (영원히 무시되는 상태가 존재하면 안됨)

Asynchronous DP

계산량이 줄어드는게 아니다.

“Of course, avoiding sweeps does not necessarily mean that we can get away with less computation. It just means that an algorithm does not need to get locked into any hopelessly long sweep before it can make progress improving a policy.”

Synchronous: 업데이트의 중요도와 무관하게 모든 상태를 균등하게 처리

Asynchronous: 중요한 상태에 집중 가능 → 동일한 계산으로 더 빠른 정책 개선

Asynchronous DP

실시간 에이전트와 결합

Synchronous DP
- 완전 수렴 후 에이전트 실행
→ 실제 문제에서 적용하기 어려움

Asynchronous DP
- 계산과 동시에 에이전트 실행

에이전트 상태 s 방문 → s 우선 업데이트 → 최신 V로 행동 결정 → …

Asynchronous DP

현실적인 효과

작은 규모 문제

→ Async DP를 이용해 보다 효율적으로 policy 개선 가능
→ In-place 구현도 엄밀하게는 Async DP의 아종

대규모 문제(ex 백개먼)

→ 모든 상태 업데이트 자체가 불가능
→ DP 자체의 한계라고 봐야 함 (이후 MC, TD로 넘어감)

GPI (Generalized Policy Iteration)

	Evaluation	**Improvement**
Policy Iteration	완전 수렴	전체 상태 한번에 업뎃
Value Iteration	1 sweep	전체 상태 한번에 업뎃
Async DP	상태 하나씩	1Evaluation 1업뎃

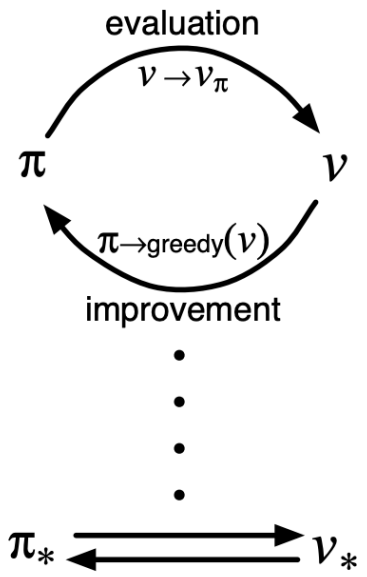
Evaluation과 Improvement가 번갈아 상호작용하며 v_*, π_* 로 수렴

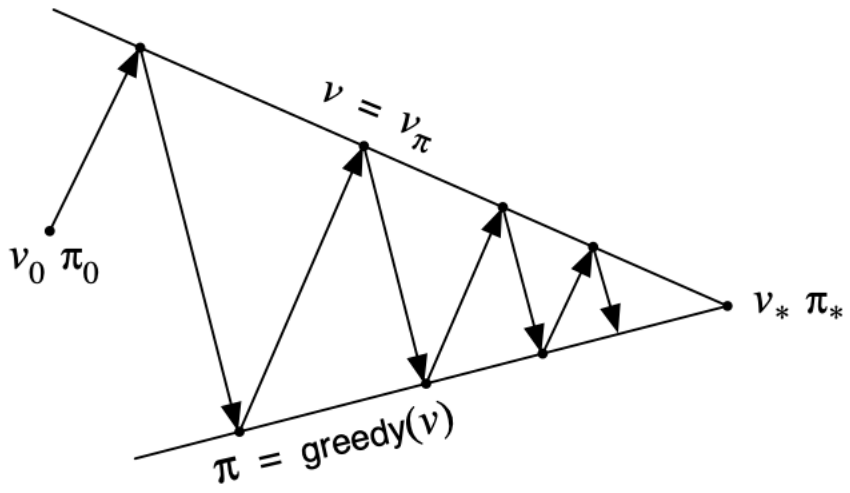
→ 이 구조가 Generalized Policy Iteration

GPI (Generalized Policy Iteration)

Evaluation과 Improvement는 경쟁관계이자 협력관계
- Policy가 바뀌면 value가 틀어지고, value가 업데이트되면 policy가 틀어짐

- 하지만 치고박고 싸우면서 Optimal 을 향해감
→ 평형을 이룬 경우 $v = v_\pi, \pi = greedy(v)$ → Bellman Optimality Eq 만족





GPI (Generalized Policy Iteration)

왜 중요한가?
- 거의 모든 RL 메소드들이 GPI로 잘 설명이 됨

Monte Carlo
- Evaluation: 에피소드 샘플로 v 추정
- Improvement: Greedy 정책 업데이트

TD Learning
- Evaluation: 한 스텝 샘플로 v 추정
- Improvement: Greedy 정책 업데이트

Actor-Critic
- Evaluation: Critic 이 v 추정
- Improvement: Actor가 정책 업데이트

Thank YOU